Міністерство освіти і науки України

Фаховий коледж зварювання та електроніки імені Є.О. Патона

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ №6

навчальної дисципліни

ПРОГРАМНІ МЕТОДИ ЗАХИСТУ ІНФОРМАЦІЇ

ФКЗЕ. 121ПМЗІ05. 06ЛР

Шифр роботи

Студент Захарчук Ю.О.

Група ПЗ-23-1/9

Перевірив Костенко А.О.

2026 - 2027

Тема: Реалізація асиметричного алгоритму шифрування RSA. Програмна реалізація.

Мета: вивчити принципи асиметричного шифрування на прикладі алгоритму RSA. Реалізувати генерацію ключів, шифрування та дешифрування.

Обладнання: ПК з компілятором або інтерпретатором мови програмування за вибором студента; Середовище розробки (Visual Studio, VS Code або інше); Калькулятор для модульної арифметики.

Варіант: 5.

Хід роботи

1. Отримати в викладача номер індивідуального варіанта.
2. Реалізувати функцію обчислення НСД (алгоритм Евкліда).
3. Реалізувати розширений алгоритм Евкліда для знаходження оберненого за модулем.

```
/// Find inversed intager in modular arithmetic
/// (num * inversed(num)) % base == 1
pub fn inversed(num: anytype, base: anytype) ?@TypeOf(num, base) {
    const T = @TypeOf(num, base);

    const ST = if (@typeInfo(T).int.signedness != .signed)
        @Int(.signed, @typeInfo(T).int.bits + 1)
    else
        T;

    const gcd, const x, _ = egcd(@as(ST, @mod(num, base)), @as(ST, base));

    return if (gcd == 1) @as(T, @intCast(@mod(x, base))) else null;
}

/// Extended Greatest Common Divisor
/// s < b
fn egcd(s: anytype, b: anytype) struct{@TypeOf(s, b), @TypeOf(s, b), @TypeOf(s, b)} {
    if (@typeInfo(@TypeOf(s, b)).int.signedness != .signed) {
        @compileError("egcd does not work with signed intagers");
    }
    if (s == 0) return .{b, 0, 1};
    assert(s < b);

    const gcd, const x1, const y1 = egcd(@mod(b, s), s);
    const x = y1 - x1 * @divTrunc(b, s);
    const y = x1;
    return .{gcd, x, y};
}
```

4. Реалізувати швидке модульне піднесення до степеня (square-and-multiply).

```
pub fn modPow(base: anytype, exp: anytype, mod: anytype) @TypeOf(base, exp, mod) {
    const T = @TypeOf(base, exp, mod);
    if (@typeInfo(T).int.signedness != .unsigned) {
```

```
        @compileError("Only unsigned type is supported");
    }
    const MT = @Int(.unsigned, @typeInfo(T).int.bits * 2); // Type for safe multiplying

    var res: T = 1;
    var b: T = base % mod;
    var e: T = exp;

    while (e > 0) {
        if (e & 1 == 1) {
            res = @intCast((@as(MT, res) * b) % mod);
        }
        e >>= 1;
        b = @intCast((@as(MT, b) * b) % mod);
    }

    return res;
}
```

5. Реалізувати генерацію ключів RSA з простими числами з варіанта.

Функція швидкої перевірки на просте число

```
pub fn isPrime(num: anytype, random: std.Random) bool {
    if (num == 2 or num == 3) return true;
    if (num < 2 or num % 2 == 0) return false;

    // num-1 as 2^r * d
    var d = num - 1;
    var r: usize = 0;
    while (d % 2 == 0) : (r += 1) {
        d /= 2;
    }

    const test_rounds = 10;

    for (0..test_rounds) |_| {
        var x = modPow(random.intRangeAtMost(@TypeOf(num), 2, num-2), // [2; num-2]
                       d, num);
        if (x == 1 or x == num - 1) continue;

        const composite = find_composite: for (0..r-1) |_| {
            x = modPow(x, 2, num);
            if (x == num - 1) {
                break :find_composite false;
            }
        } else true;

        if (composite) return false;
    }
    return true;
}
```

Функція генерації простих чисел

```
pub fn genPrime(T: type, random: std.Random) T {
    const bits = @typeInfo(T).int.bits;

    while (true) {
        const candidate = random.intRangeAtMost(
            T, @as(T, 1) << (bits-1), ~@as(T, 0)
        ) | 1; // make odd
        if (isPrime(candidate, random)) return candidate;
    }
}
```

Функція генерація пари ключів

```
pub fn genKeys(pub_sink: *Io.Writer, priv_sink: *Io.Writer) Io.Writer.Error!void {
    var rng = std.Random.DefaultPrng.init(
        @bitCast(@as(i64, @truncate(std.Io.Timestamp.now(global.io, .real).nanoseconds)))
    );
    const random = rng.random();

    // Generate 2 prime numbers
```

```
    const p = util.genPrime(Prime, random);
    var q = util.genPrime(Prime, random);
    while (q == p) q = util.genPrime(Prime, random);

    const n: Key = @as(Key, p) * q; // Part of public and private key
    const phi: Key = @as(Key, p - 1) * (q - 1);

    // Open exponent
    // var e: Key = 65537; // Part of public key
    // if (e >= phi) e = 17;
    // while (std.math.gcd(e, phi) != 1) e += 2;

    // Generate maximum size open exponent, embrase chaos!
    var e = phi - 1;
    while (std.math.gcd(e, phi) != 1) e -= 1;

    // The only private part of key
    // We have e as part of public key, so the only thing keeping us private is phi
    // for which to get from n we need to find its simple multipliers, which is very
   expensive,
    // so we must be safe.
    const d = util.inversed(e, phi).?; // Part of private key

    try pub_sink.writeInt(Key, n, .big);
    try pub_sink.writeInt(Key, e, .big);

    try priv_sink.writeInt(Key, n, .big);
    try priv_sink.writeInt(Key, d, .big);
}
```

6. Реалізувати шифрування: C = M^e mod n.

```
pub fn encrypt(pub_key: []const u8, source: []const u8, sink: *Io.Writer)
   Io.Writer.Error!bool {
    const n, const e = readKey(pub_key) orelse return false;

    var iter: BlockIter((@typeInfo(Key).int.bits / 8) / 2) = .init(source);
    while (iter.next()) |block| {
        const C: Key = util.modPow(@as(Key, block), e, n);
        try sink.writeInt(Key, C, .big);
    }

    return true;
}
```

7. Реалізувати дешифрування: M = C^d mod n.

pub fn decrypt(priv_key: []const u8, source: []const u8, sink: *Io.Writer) Io.Writer.Error!bool { const n, const d = readKey(priv_key) orelse return false;

```
pub fn decrypt(priv_key: []const u8, source: []const u8, sink: *Io.Writer)
   Io.Writer.Error!bool {
    const n, const d = readKey(priv_key) orelse return false;

    var iter: BlockIter(@typeInfo(Key).int.bits / 8) = .init(source);
    while (iter.next()) |block| {
        const P: Key = util.modPow(block, d, n);
        const Pblock = std.math.cast(Prime, P) orelse {
            print("Failed to cast intager to block size. Probably key is wrong\n", .{});
            return false;
        };
        try sink.writeInt(Prime, Pblock, .big);
    }

    return true;
}
```

1. Перевірити роботу алгоритму на числовому прикладі.
2. Реалізувати шифрування текстового повідомлення.

```
[~/Documents/College/crypt/crypt] <yurii> ./crypt -g=rsa rsa_ke

[~/Documents/College/crypt/crypt] <yurii> ls rsa*
rsa_key  rsa_key.pub
```

Рисунок 1 - Генерація пари ключів

```
[~/Documents/College/crypt/crypt] <yurii> ./crypt -e=rsa rsa_key.pub sentence.txt sentence.rsa_cryp

Operation succeed

[~/Documents/College/crypt/crypt] <yurii> xxd sentence.rsa_crypted
00000000: 12c3 2742 86cf 36b4 1280 7a34 bbc2 dda5  ..´B..6...z4....
00000010: f437 0250 f5aa 3f58 b135 805d 747c e9c3  .7.P..?X.5.]t|..
00000020: 344b c352 9ba6 8f02 5e21 a4b8 b230 f878  4K.R....^!...0.x
00000030: 8373 5d69 e1af 8a85 530a 8277 b550 8b8b  .s]i....S..w.P..
00000040: 151f b53e ea97 8c81 7b4a dd5f b4c1 2d78  ...>....{J._..-x
00000050: 2adf 611c cbef 4d22 23c1 b711 c3db 8317  *.a...M"#.......
00000060: 9d26 c5c2 022b 7fac a6ab 3ad7 6109 0fb6  .&...+....:.a...
00000070: 28c8 2246 e2d4 54e4 04d0 4e6e 2dcf 7ef8  (."F..T...Nn-.~.
00000080: 9a40 17f5 1180 523d 917b 3341 f126 0387  .@....R=.{3A.&..
00000090: 1193 bb75 52a5 9d5d 25b1 0c8c bc5f 5643  ...uR..]%...._VC
000000a0: b73a 4ae8 dc14 c31c efd6 e800 2762 6c8c  .:J.........´bl.
000000b0: e949 2a9f 6f69 ad23 e376 b6d4 faf1 9b6a  .I*.oi.#.v.....j
000000c0: 91fb 03d0 5e0b 8fe1 d7f2 b996 46bb 2140  ....^.......F.!@
000000d0: 0d5b ef10 ffb6 33d6 4f74 1c72 6450 88c9  .[....3.Ot.rdP..
000000e0: 3331 1fc3 48a6 70b8 0189 e19a 455c 2f8d  31..H.p.....E\/.
000000f0: 108a 41fd 9289 31a3 9834 732c 6430 70fa  ..A...1..4s,d0p.
00000100: 62a0 72c2 1c67 e39b 4e20 e3ba f8ba 4496  b.r..g..N ....D.
00000110: d6fe d7f5 3aca 5808 4e59 27dd ee59 ee2a  ....:.X.NY´..Y.*
00000120: 9dd0 dc10 4a8e a914 9ff1 779a 348d f9a4  ....J.....w.4...
00000130: f9f6 ea04 cdbf 282e 35e1 54f9 3d5e b7c8  ......(.5.T.=^..
00000140: 92cb ef16 8366 833a 47a2 6352 c28e fde8  .....f.:G.cR....
00000150: c522 1aeb 38c9 f1e4 7062 1b4f a0e3 64f0  ."..8...pb.O..d.
00000160: a701 9fde 4942 98f2 bfb7 152c 259d 017f  ....IB.....,%...
00000170: b050 f047 d1f6 f0da e3c3 15c7 45a5 d043  .P.G........E..C
00000180: 58da dbfb 14d2 d1a6 8bf3 c944 1d89 4611  X..........D..F.
00000190: 83da dfff 85fe 415f 7bf4 9a04 ac80 6c62  ......A_{.....lb
000001a0: f38a 02bb f470 95bf 21d4 c3f7 d065 87a5  .....p..!....e..
000001b0: 1640 b2c9 9716 3d44 3961 ad0c fab7 380d  .@....=D9a....8.
000001c0: 96a4 ca51 a9dc 4e35 e174 6b35 6083 1a31  ...Q..N5.tk5`..1
000001d0: 6240 b866 c07b 37a5 7304 24d1 022a c699  b@.f.{7.s.$..*..
000001e0: 7dc9 c16f 1214 dcd5 336b 83c8 4542 8d49  }..o....3k..EB.I
000001f0: 4a23 5f63 5893 e04c c285 8891 f3e3 a580  J#_cX..L........
```

Рисунок 2 - Шифрування файлу

```
[~/Documents/College/crypt/crypt] <yurii> ./crypt -d=rsa rsa_key sentence.rsa_crypted

The morning was quiet except for the small mechanical hum of the city waking up. Light sl
across the windows in thin gold lines.

Operation succeed
```

Рисунок 3 - Дешифрування файлу

3. Виконати індивідуальне завдання згідно з варіантом.

```
./crypt -e=rsa rsa_key_17257.pub sentence.txt sentence.rsa_crypted  0.01s user 0.00s system 96% cpu 0.013 to
```

Рисунок 4 - Час шифрування із стандартною експонентою

```
./crypt -e=rsa rsa_key_max.pub sentence.txt sentence.rsa_crypted  0.06s user 0.00s system 99% cpu 0.058 to
```

Рисунок 5 - Час шифрування із e = 17 257

```
./crypt -e=rsa rsa_key_max.pub sentence.txt sentence.rsa_crypted  0.06s user 0.00s system 99% cpu 0.060 to
```

Рисунок 6 - Час шифрування із максимальною експонентою

4. Оформити звіт та зробити висновок.

Відповіді на контрольні питання

Високий рівень (творчий)

1. Поясніть, чому зростання довжини ключа RSA дає нелінійний приріст стійкості, і оцініть, чому 1024 біти вже вважають недостатніми.

   Основа RSA - складність розкладання числа на прості множники. Такий алгоритм є дуже складним, бо для цього потрібно не просто перебирати ключі, а використовувати складну математику, тому й приріст стійкості від розміру ключа нелінійний - це не просто n нових значень для брут-форсу, а ціла поліада нових віток, які треба обчислити.

   Не дивлячись на це, комп'ютери стають все потужнішими, і ключ у 1024 бітів вже не вважається неможливим для розкладання, тому мінімум збільшується.
2. Запропонуйте порядок дій для безпечної генерації ключової пари в навчальній лабораторії та поясніть роль джерела випадковості.
   1. Взяти реалізацію алгоритму
   2. Згенерувати ключі

   Роль джерела випадковості: якщо можливо визначити та відтворити, як були згенеровані числа для ключів, то це є великою вразливістю (наприклад, класично у C як зерно використовується поточний час у секундах, у такому випадку якщо хтось дізнається час генерації ключа, то він зможе його відтворити)
3. Порівняйте RSA й криптографію на еліптичних кривих за довжиною ключа, швидкодією та поширеністю.

Критерій	RSA Еліптичні криві (ECC)
Розмір ключа Швидкодія Поширеність	Для приблизно 112 бітів стійкості — RSA-2048; для 128 — RSA-3072 Для приблизно 128 бітів — крива з параметром 256 бітів, наприклад P-256 Операції з відкритим ключем часто швидкі; генерація ключів і операції з приватним ключем можуть бути повільнішими, особливо зі збільшенням ключа Ключі й підписи компактніші; швидкість залежить від конкретного алгоритму, кривої, бібліотеки та платформи Дуже широко використовується у наявних системах і сумісний із багатьма старими протоколами Також широко поширена в сучасних протоколах; часто обирається, коли важливі менший розмір ключів і менше навантаження на мережу

4. Сформулюйте, які загрози для RSA створює поява квантових обчислень, і які напрями розв'язання цієї проблеми розвиваються.

За рахунок магії квантової фізіки з’явився алгоритм, що може фактично безкоштовно розшифровувати RSA.

Для розв’язання цієї проблеми зараз активно розробляються і починають застосовуватися нові алгоритми, стійкі до квантових комп’ютерів: ML-KEM, ML-DSA, SLH-DSA та інші

Висновок: Я вивчив основні принципи асиметричного шифрування, практично реалізував їх у вигляді алгоритму RSA